Laboratory 4: REST API

Name: Aron Jay A. Formento
Role: SQL Developer
Tasks Performed

- Designed the database schema and wrote the SQL queries that the backend used.

Code Contributions

```sql
-- ========================================================
-- TABLE 3: Bus Routes (Junction Table)
-- Links buses to routes with custom capacity per route
-- Enables M:N relationship between buses and routes
-- ========================================================
CREATE TABLE bus_routes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    bus_id INT NOT NULL,
    route_id INT NOT NULL,
    capacity INT NOT NULL COMMENT 'Capacity override for this specific route',
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (bus_id) REFERENCES buses(id) ON DELETE CASCADE,
    FOREIGN KEY (route_id) REFERENCES routes(id) ON DELETE CASCADE,
    UNIQUE KEY unique_bus_route (bus_id, route_id),
    INDEX idx_bus_id (bus_id),
    INDEX idx_route_id (route_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

b

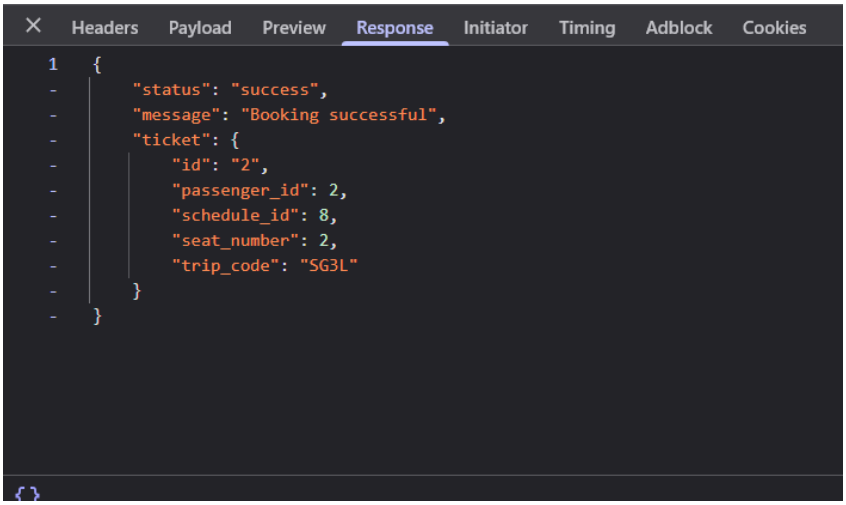
This particular SQL snippet creates a junction table called bus_routes that establishes a **many-to-many relationship** between buses and routes by linking their respective IDs. It ensures data integrity through **Foreign Keys**, which use ON DELETE CASCADE to automatically remove assignments if a parent bus or route record is deleted. The table includes a capacity field to allow for route-specific passenger limits and a **unique composite key** to prevent the same bus from being assigned to the same route twice. This structure is essential for the API, as it allows the system to efficiently query and return combined transportation data as a single JSON object.

Challenges & Problem Solving

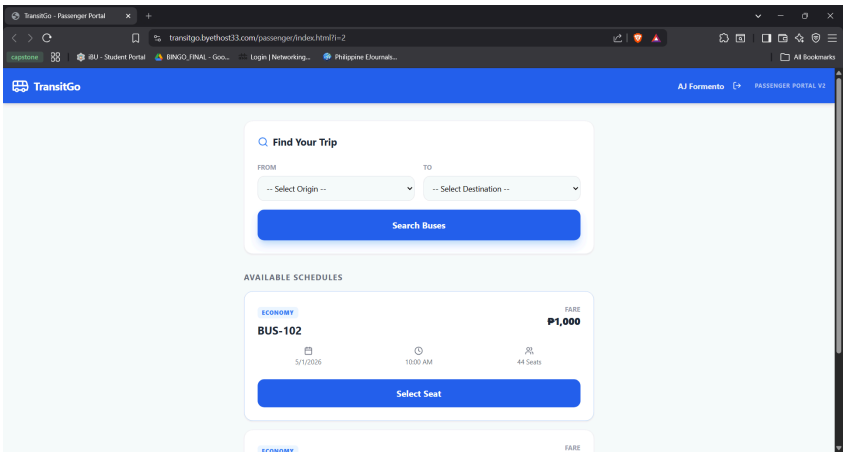
A major challenge we faced was that our data was spread across four different tables (**Buses**, **Routes**, **Bus_Routes**, and **Schedules**). Initially, when the Client application requested trip details, the API would only return raw ID numbers (e.g., bus_id: 101, route_id: 5). This was a problem because the user needs to see the actual 'Bus Number,' 'Origin,' and 'Destination' names, not just the IDs.

Solution:
To solve this, we implemented an advanced **SQL JOIN** query within the backend PHP script. By joining the schedules table with the bus_routes, buses, and routes tables, we were able to fetch all descriptive names in a single database call. We then structured the PHP output to nest this data into a clean **JSON object**. This ensured that the Client applications received a complete, human-readable package of information (e.g., 'origin': 'Manila') without having to make multiple separate API requests, which significantly improved the app's performance and user experience."

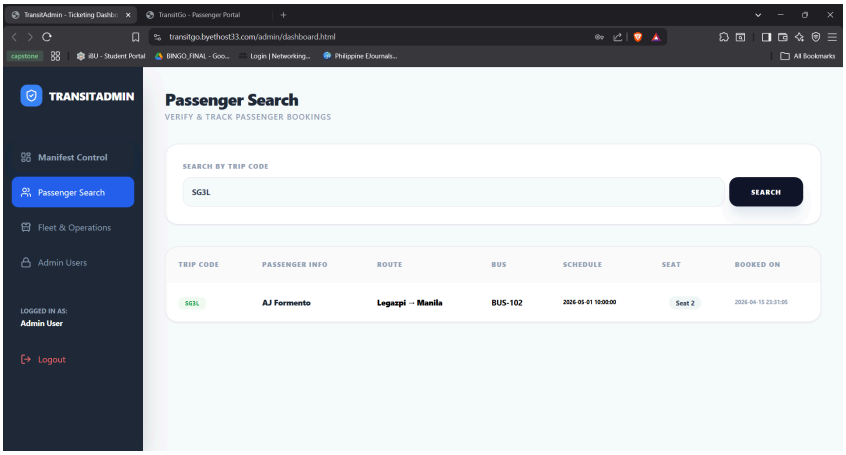
5. Evidence & Screenshots



API Response: Ticket Booking



Client Application UI: Passenger Booking Interface



Client Application UI: Admin Dashboard Interface